PROJETO FINAL · GRUPO 2

RICK AND MORTY

Da decisão arquitetural ao código

React Native · Expo · API REST · Arquitetura em camadas

APRESENTAÇÃO TÉCNICA • 10min50s

Problema, objetivo e escopo

Sete integrantes, sete funcionalidades e desenvolvimento paralelo

Objetivo

Consultar a API pública de Rick and Morty com navegação fluida e experiência consistente.

Condições do projeto

Prazo curto, entrega Android, tratamento de carregamento, erro e ausência de resultados.

Organização da equipe

Uma feature por integrante, branches independentes e integração por pull request.

7 FEATURES

3 ABAS PRINCIPAIS

API PÚBLICA

Decisões técnicas: por que esta stack?

Escolhas registradas em uma ADR aceita em 04/11/2026

React Native + Expo

Reaproveitar experiência da equipe e o projeto-base.

TanStack Query

Dados da API: cache, paginação, carregamento e erros.

Zustand + AsyncStorage

Preferências locais: favoritos e tema persistidos.

Paper + React Navigation

Componentes consistentes e navegação por abas/detalhes.

Alternativas avaliadas: Flutter/KMP, fetch + useEffect, Context/Redux, MMKV e Expo Router.

ADR: contexto e consequências

Decisão registrada em rickandmorty-rn/ADR.md · status: aceita em 04/11/2026

CONTEXTO

7	5	7
pessoas	dias	features

O que mais pesou: trabalhar em paralelo sem um travar o outro, com quase todo o estado vindo da API.

O que ganhamos

- Cada um dono dos próprios arquivos, com contratos prontos no primeiro dia.
- Carregando, erro e vazio vêm do estado da query em todas as telas.
- Testes em funções puras, sem React e sem rede.

Custo assumido

- Duas bibliotecas de estado para aprender e explicar.
- Busca e filtro dependem da rede: sem internet, só o cache e os favoritos.

A regra que tornou o custo aceitável: **veio da API → TanStack Query · escolha do usuário → Zustand**

ADR: alternativas descartadas

O que o grupo considerou e por que não escolheu

Flutter ou KMP

O grupo inteiro trocaria de stack a cinco dias do prazo.

fetch + useEffect

Sete telas repetindo loading, erro, retry e paginação à mão.

Só Zustand

Reescrever cache e paginação que o TanStack Query já resolve.

Context + useReducer

Verboso, e cada mudança renderiza de novo todos os consumidores.

Redux Toolkit

Resolve o mesmo problema com mais cerimônia do que o app pede.

MMKV

Exige módulo nativo; com dezenas de favoritos, a diferença não aparece.

Expo Router

Já tínhamos React Navigation; migrar não muda nada para o usuário.

Expo SDK 58

Mudaria React Native e bibliotecas para os sete a cinco dias do prazo.

Critério comum: nada que exigisse reaprender ou migrar algo para os sete com o prazo de cinco dias.

Arquitetura: cada camada com uma responsabilidade

A interface não realiza requisições HTTP diretamente

SCREENS — Telas e interação

↓

QUERIES / STORES — Estado remoto / local

↓

API — HTTP e conversão de DTO

↓

DOMAIN — Tipos e regras puras

Estrutura de src/

api/ · domain/ · queries/ · stores/
theme/ · components/
screens/ · navigation/

Componentes reutilizáveis e regras testáveis reduzem acoplamento.

As sete funcionalidades

Entregas divididas entre os integrantes

F1 — **Lista e scroll infinito**
Paulo

F2 — **Detalhe e compartilhamento**
Vinícius

F3 — **Busca + vazio/erro/offline**
Hugo

F4 — **Filtro por status**
Guilherme

F5 — **Favoritos persistidos**
Marcus

F6 — **Episódios e relações**
Rafael

F7 — **Tema claro/escuro**
Sesaque

Fluxo do código: lista, busca e filtro

Uma única consulta recebe os filtros e controla páginas

```
// Assinaturas e fluxo documentados no projeto

useCharacters({ name, status })

GET /character?page=N&name=...&status=...

useInfiniteQuery  →  cache por filtros
```

Por que assim?

Busca e filtro podem ser combinados. A paginação ocorre na API, e o cache preserva resultados por combinação.

404 sem resultados → estado vazio, e não falha genérica.

Detalhes, episódios e favoritos

Integração entre telas e persistência local

Detalhe

Exibe informações do personagem, imagem, origem e localização; prevê compartilhamento e favoritos.

Episódios

Organiza episódios por temporada e conecta personagens aos episódios em que aparecem.

Favoritos

Zustand + persist + AsyncStorage salvam IDs e informações essenciais para uso offline.

Regra central: dados vindos da API → Query | escolhas do usuário → Zustand

UI/UX: consistência entre sete entregas

Design system compartilhado e estados previstos

Tokens

Cores, tipografia e espaçamento centralizados

Componentes

Cards, badges, skeleton e feedbacks reutilizáveis

Temas

Claro, escuro e preferência armazenada

Resiliência

Carregando, vazio, erro e rede indisponível

Resultado esperado: experiência uniforme, mesmo com desenvolvimento em paralelo.

Qualidade: Git, testes e integração

Processo de trabalho registrado no repositório

01 Branch por feature → **02** Implementação + teste → **03** Pull request + revisão → **04** CI / typecheck / build → **05** Integração na main

Pontos de validação

Funções puras, tipagem TypeScript e build Android; revisão antes de integrar alterações.

Conclusão: o que aprendemos

Uma arquitetura simples foi o principal facilitador do trabalho em equipe

Principal acerto

Separação clara de responsabilidades e contratos compartilhados para desenvolver em paralelo.

Trade-offs conscientes

Mais bibliotecas para explicar, dependência de rede para buscas e compromisso entre prazo e robustez.

DO PLANEJAMENTO → DECISÕES → CÓDIGO → INTEGRAÇÃO

Repositório: github.com/aranha/mobile-2026-2-grupo-2-rickandmorty

BAIXE O APP · ANDROID

Aponte a câmera do celular

Release v1.0.0 · APK de 56 MB